REQUIREMENTS & TECHNICAL SPECIFICATION

E-commerce Store: Requirements & Technical Specification

Version 2.1 (Final) · 24 October 2026 · Next.js + FastAPI · 12-week build

Document owner	Tanvir Hasan
Audience	Development team
Business model	Single-vendor online store: the owner holds all stock and sells only through this website
Version	2.1 (Final), supersedes Versions 2.0 and 1.0 of 24 October 2026
Status	Final, approved for build. Open decisions in section 2 must be confirmed before Week 1.
Priority key	M must S should C could NEW added since v1.0 UPDATED changed since v1.0

Changes from v1.0

Version 2.1 is the final version for the build. It applies the v2.0 changes and then the v2.1 single-vendor revision; v2.1 takes precedence where the two differ. Requirements that are new or changed since v1.0 are marked NEW or UPDATED in section 5.

Ver.	Where	Change
2.1	Whole document	**Single-vendor.** The owner holds all stock and sells only on this website. Title changed to 'E-commerce Store'. Roles reduced to Guest, Customer, Staff (support/warehouse, permissions set by the super admin) and Super admin (owner); the moderation role became an optional Staff permission. Third-party vendor onboarding, the vendor portal, fee and disbursement features, and order splitting removed. Product, stock, order and fulfilment management moved into the admin back office (5.8). Single order-level state machine (9.4). Simple revenue/refund ledger (FR-PAY-6). Customer-support messaging and tickets to the store (5.13). Admin sales analytics. Plan rebalanced to 12 weeks with buffer. Diagrams, data model, API, tests, risks and CV bullet updated.
2.0	Section 1, cover	Typo fixed ('ecomerse' to 'e-commerce'); build length stated in the subtitle (now 12 weeks, see 2.1).
2.0	5.1 FR-AUTH-1, 10	Phone-registered users verify their phone by OTP; checkout requires a verified email **or** a verified phone. Phone verification endpoints added.
2.0	5.1 FR-AUTH-6, 7.1, 16	2FA made consistent: mandatory for the super admin and all staff, optional for customers.
2.0	5.5 FR-ORD-1, 9.4	State machine clarified: online payment (pending_payment to paid) and COD (pending_payment to confirmed) separated; delivery_failed, returned and return_rejected added; partial refunds per order item.
2.0	5.6 FR-PAY-8, 9.5, 7.10, 13	Late-webhook handling: payment confirmed after reservation expiry or cancellation re-reserves stock and reinstates the order, otherwise auto-refunds and notifies. Covered by tests.
2.0	7.5, 9.5, 10	Gateway return/callback endpoints never depend on session cookies (SameSite=Lax POST redirect); verification through the gateway validation API only.
2.0	5.6 FR-PAY-3, 7.5	'Processed exactly once' reworded to 'processed effectively once via deduplication on gateway event id (unique constraint)'.
2.0	2	Open decisions: bKash/SSLCommerz are Bangladesh-only; target country decided first (e.g. Bangladesh vs Hungary/EU with Stripe/Barion); legal entity and owner eligibility confirmed.
2.0	4, 5, 8, 9, 10	New requirements: customer-support messaging (S), COD/new-account fraud risk scoring (S), feature flags (S), admin impersonation with audit log and consent (C), price history (C), Bangla/English i18n-ready UI (C), installable PWA storefront (C).
2.0	14, 15	Original 8-week plan replaced (now a 12-week plan with buffer, see 2.1); scope-cut order and risks updated; 'never cut' list kept unchanged.
2.0	Front matter	This changelog and a contents page added.

Contents

1. Purpose and goals 4
2. Scope, assumptions, open decisions 4
3. Users and roles 5
4. Technology stack 6
5. Functional requirements 7
6. Non-functional requirements 13
7. Security requirements 13
8. UI and UX requirements 17
9. Architecture and data model 19
10. API conventions 24
11. RAG assistant design 25
12. DevOps, environments and observability 26
13. Testing strategy 27
14. Twelve-week plan 28
15. Risks and mitigations 30
16. Definition of done and launch checklist 30
17. Portfolio and CV plan 31
18. Next steps 31

1. Purpose and goals

Business goal. A production-grade, single-vendor e-commerce store that the owner will run as a real business. The owner holds all stock and sells only through this website: customers browse and buy, the owner and staff manage the catalog, stock and fulfilment from the admin back office, and an AI assistant (RAG) helps with shopping and support.

Career goal. The same codebase is the flagship portfolio project for senior engineering roles. Decisions are therefore documented (ADRs), measured (load tests, RAG evaluation) and defended (security, tests, observability).

Success criteria

- A customer completes search, cart, checkout, payment and order tracking without errors on mobile and desktop.
- No overselling under concurrent checkout (proven by a load test).
- No double charge or lost payment (idempotency plus webhook reconciliation, including late webhooks).
- The security baseline in section 7 is reviewed and passed before real payments go live.
- Core Web Vitals rated 'good' on listing and product pages.
- The RAG assistant answers from catalog and policy data, with measured faithfulness.

2. Scope, assumptions, open decisions

In scope for 12 weeks: every requirement marked Must (M) or Should (S) in section 5. Could (C) items are scheduled in Week 10 and are the first to be cut (section 14).

Deferred (Could/Won't for v1): native mobile apps (the installable PWA, FR-PWA-1, covers home-screen installs), multi-language UI beyond the Bangla/English i18n-ready UI (FR-I18N-1), more than one live courier integration, advertising/sponsored products, loyalty points, splitting into more than three services, selling on third-party channels.

Assumptions (correct me if wrong)

1. Single-vendor: the owner owns all stock and sells only through this website. The owner is the super admin; staff accounts are added as the business grows.
2. One country and one currency at launch (currency is configurable; local gateway assumed).
3. Payments use a hosted gateway redirect plus Cash on Delivery. The gateway depends on the target country: for example SSLCommerz or bKash for Bangladesh, Stripe or Barion for Hungary/EU. Stripe is an optional international path. Card data never touches our servers.
4. One developer and 12 weeks, so a modular monolith, not many microservices.
5. The LLM is reached through a provider API; the embedding model is swappable.

Decisions the owner must confirm before Week 1

> **Decide the target country first.** It determines the payment gateway, currency, tax/VAT rules, privacy regime and legal entity. bKash and SSLCommerz serve Bangladesh only and cannot be used for a

> Hungary/EU launch. A Hungary/EU launch would use, for example, Stripe or Barion (EUR or HUF) with EU VAT and GDPR; a Bangladesh launch would use bKash and/or SSLCommerz in BDT.

- **Target country** (for example Bangladesh vs Hungary/EU), then currency, tax/VAT rules, and the payment gateway that fits that country.
- **Legal entity and owner eligibility:** confirm that the legal entity that onboards with the payment gateway, and the owner behind it (company registration, residency, local bank account, required documents), are eligible for the chosen gateway in the chosen country.
- Return and refund policy, delivery partners, delivery charge rules.
- Whether Cash on Delivery is allowed, and for which categories.
- Hosting budget and region.

3. Users and roles

Role	Description	Key permissions
Guest	Unauthenticated visitor	Browse, search, view products, use chatbot (limited)
Customer	Registered buyer	Cart, checkout, orders, reviews, wishlist, addresses, returns, support tickets
Staff	Internal: support and/or warehouse	Permissions set by the super admin per staff account, for example support (tickets, customers, orders, refunds up to a limit), warehouse (stock, picking, packing, shipping, returned parcels), and optional moderation (reviews, product Q&A, risk review queue); impersonate a customer (read-only, with consent, audited; Could)
Super admin	Owner	Everything: catalog, stock, orders, refunds, configuration, staff and permission management, feature flags, analytics

Authorization is enforced on the server for every request using role, explicit staff permissions, and resource ownership (a customer can only touch their own orders, addresses, tickets and reviews). The super admin and every staff account must use two-factor authentication.

4. Technology stack

Layer	Choice	Reason
Frontend	Next.js (App Router) + TypeScript	SSR/ISR for SEO and speed, strong ecosystem
UI	Tailwind CSS + shadcn/ui + Radix	Accessible primitives, consistent design system
Client data	TanStack Query, Zustand, React Hook Form + Zod	Server-state caching, light client state, typed validation
Backend	FastAPI, Pydantic v2, SQLAlchemy 2.0 (async), Alembic	Typed, fast, auto OpenAPI
Database	PostgreSQL	Transactions, row locking, JSONB, full-text, pgvector
Cache/queue	Redis + ARQ or Celery	Cache, rate limit, carts, locks, background jobs
Search	Meilisearch or OpenSearch	Typo tolerance, facets, autocomplete
Vector store	pgvector (start), Qdrant (optional)	One less system to run at first
Storage/CDN	S3-compatible (R2/S3) + CDN	Images and invoices
Messaging	Redis Streams or RabbitMQ	Domain events, outbox relay
Observability	OpenTelemetry, Prometheus, Grafana, Sentry, structured JSON logs	Debuggability in production
Delivery	Docker, GitHub Actions, Terraform (basic), reverse proxy (Nginx/Caddy)	Repeatable deploys
Testing	pytest, Vitest, Playwright, k6	Unit, integration, E2E, load
Feature flags NEW	Own flags table with cached evaluation (OpenFeature-compatible interface)	Gradual rollout and kill switches without redeploys
i18n / PWA NEW	next-intl (or equivalent); web app manifest + service worker	Bangla/English-ready UI; installable storefront

5. Functional requirements

Priority: M = must, S = should, C = could. NEW = added since v1.0, UPDATED = changed since v1.0. Requirement IDs were renumbered in v2.1 where items were removed or merged.

5.1 Authentication and accounts

ID	Requirement	P
FR-AUTH-1	Register with email or phone plus password. Email-registered users verify their email (single-use link); phone-registered users must verify their phone by SMS OTP. Checkout requires at least one verified contact: a verified email **or** a verified phone. UPDATED	M
FR-AUTH-2	Login with short-lived access token and rotating refresh token in httpOnly secure cookie	M
FR-AUTH-3	Password reset by single-use, time-limited link	M
FR-AUTH-4	Google OAuth login	S
FR-AUTH-5	OTP by SMS for phone verification and COD confirmation	S
FR-AUTH-6	Two-factor authentication (TOTP) mandatory for the super admin and all staff accounts; optional for customers UPDATED	M
FR-AUTH-7	Session list and 'log out everywhere'	S
FR-AUTH-8	Address book, profile, account deletion/export (privacy)	M

> FR-AUTH-1 is Must, so OTP verification is required for phone-registered users even though SMS OTP in general (FR-AUTH-5) is Should. If SMS is cut, registration is limited to email.

5.2 Catalog

ID	Requirement	P
FR-CAT-1	Hierarchical categories with attributes per category	M
FR-CAT-2	Products with variants (size, colour), SKUs, price, compare-at price, images, description, specs	M
FR-CAT-3	Multiple images with automatic resize and WebP/AVIF	M
FR-CAT-4	Product publishing workflow: draft, in review (optional second-person check by staff), published, archived UPDATED	M
FR-CAT-5	Brand pages, collection pages, product slug URLs UPDATED	S
FR-CAT-6	Bulk import/export by CSV	S
FR-CAT-7	Price history on the product page: per-variant price chart (for example last 90 days) and the lowest price of the previous 30 days shown next to any discount; recorded append-only from PriceChanged events NEW	C

> If the target is Hungary/EU, EU rules on announcing price reductions refer to the prior 30-day lowest price, so FR-CAT-7 data also supports compliance; confirm in the legal review (section 2).

5.3 Search and discovery

ID	Requirement	P
FR-SRCH-1	Full-text search with typo tolerance, filters (price, brand, rating, category, attributes) and sorting	M
FR-SRCH-2	Autocomplete and recent searches	S
FR-SRCH-3	Search index updated by events within seconds of a product change	M
FR-SRCH-4	Recommendations: related products, 'customers also bought'	C
FR-SRCH-5	Recently viewed products	S

5.4 Cart and checkout

ID	Requirement	P
FR-CART-1	Cart for guests (cookie) merged into account on login	M
FR-CART-2	Price and stock re-validated at checkout; changes shown to the user	M
FR-CART-3	Coupons and vouchers applied with clear rules (min spend, category, expiry, usage limit)	S
FR-CART-4	Delivery charge calculated by zone and weight	M
FR-CART-5	Guest checkout	C

5.5 Orders and fulfilment

ID	Requirement	P
FR-ORD-1	Single order-level state machine (section 9.4): pending_payment, paid, confirmed, packed, shipped, delivered, delivery_failed, returned, completed, cancelled, return_requested, return_rejected, refunded; one order per checkout, one payment per order; refund_status (none, partial, full) tracked alongside UPDATED	M
FR-ORD-2	Customer can cancel before shipping; staff can cancel with reason UPDATED	M
FR-ORD-3	Tracking page with status timeline	M
FR-ORD-4	Return and refund flow with reason, photos, approval or rejection with reason, partial refund per order item UPDATED	S
FR-ORD-5	PDF invoice generation and email	S
FR-ORD-6	Every state change recorded in an immutable order event log	M
FR-ORD-7	Failed delivery handling: courier failure or COD refusal moves the order to delivery_failed, then (after allowed re-attempts) returned, then cancelled with stock restocked and any prepaid amount refunded NEW	M

5.6 Payments

ID	Requirement	P
FR-PAY-1	Hosted gateway payment, gateway chosen per target country (for example SSLCommerz/bKash for Bangladesh, Stripe/Barion for Hungary/EU), with server-side verification of every payment UPDATED	M
FR-PAY-2	Cash on Delivery with confirmation rules	M
FR-PAY-3	Idempotency keys on payment creation; webhooks verified by signature and processed effectively once via deduplication on gateway event id (unique constraint) UPDATED	M
FR-PAY-4	Nightly reconciliation job comparing gateway records with our records	S
FR-PAY-5	Refunds through the gateway (or manual for COD), full or partial per order item, recorded in the ledger UPDATED	S
FR-PAY-6	Simple revenue and refund ledger: append-only entries for captured payments, COD cash collected, refunds and gateway fees, per order; daily totals for accounting and reconciliation UPDATED	S
FR-PAY-7	Fraud risk scoring for COD and new accounts: rule-based score at checkout (account age, verified contacts, order value, COD refusal history, phone/address/device reuse, velocity, blocklist); thresholds decide allow, require OTP, prepaid only, or hold for manual review; decisions and reasons stored and auditable NEW	S
FR-PAY-8	Late payment confirmation: a verified payment that arrives after the reservation expired or the order was cancelled re-reserves stock and reinstates the order; if any line cannot be re-reserved, the payment is refunded automatically and the customer notified (section 9.5). Covered by automated tests NEW	M

5.7 Inventory

ID	Requirement	P
FR-INV-1	Stock per SKU; reservation at checkout with expiry (for example 15 minutes)	M
FR-INV-2	Oversell prevention using atomic updates or row locks	M
FR-INV-3	Low-stock alerts to the super admin and warehouse staff UPDATED	S
FR-INV-4	Flash-sale support with Redis-backed counters	C

5.8 Admin back office: catalog, stock and fulfilment UPDATED

All product, stock, order and fulfilment management happens in the admin back office, used by the super admin and staff according to their permissions.

ID	Requirement	P
FR-OPS-1	Product, variant, image and stock management (including stock adjustments with reason, and stock receipts)	M

ID	Requirement	P
FR-OPS-2	Order fulfilment: confirm, pick and pack (packing slip), ship, enter tracking, record delivery failures and returned parcels	M
FR-OPS-3	Return handling: approve or reject with reason, receive and inspect items, restock or write off, trigger refunds UPDATED	M
FR-OPS-4	Admin sales analytics: revenue, orders, average order value, top products and categories, conversion, refund and return rates UPDATED	S
FR-OPS-5	Customer questions and review replies from the store	S

5.9 Admin and configuration

ID	Requirement	P
FR-ADM-1	Manage customers, staff accounts and staff permissions, categories, banners, homepage sections UPDATED	M
FR-ADM-2	Moderation queue for reviews and product Q&A (optional staff permission) UPDATED	M
FR-ADM-3	Order, refund, return and complaint handling UPDATED	M
FR-ADM-4	Delivery zones, tax and coupon configuration UPDATED	M
FR-ADM-5	Dashboard: revenue, orders, conversion, top products, failure rates UPDATED	S
FR-ADM-6	Searchable audit log of all admin and staff actions UPDATED	M
FR-ADM-7	Feature flags: on/off and percentage flags with targeting (role, user, environment); kill switches for risky features (a payment method, chatbot, support messaging); every change audit logged; server-side evaluation with a short cache NEW	S
FR-ADM-8	Admin impersonation ('view as customer') for support: recent re-authentication and 2FA, recorded customer consent (or consent captured on a linked support ticket), read-only by default, time-limited, persistent on-screen banner; cannot view or change passwords, 2FA or payment details; start, end and every action audit logged with both actor and subject NEW	C

5.10 Reviews and engagement

ID	Requirement	P
FR-REV-1	Reviews and ratings only from verified purchasers	M
FR-REV-2	Review images and moderation	S
FR-REV-3	Wishlist and back-in-stock notification	S
FR-REV-4	Product Q&A	C

5.11 Notifications

ID	Requirement	P
FR-NOT-1	Transactional email: verification, order, shipping, refund	M
FR-NOT-2	SMS for OTP and delivery updates	S
FR-NOT-3	In-app notification centre	C
FR-NOT-4	User notification preferences and unsubscribe for marketing mail	M

5.12 AI shopping and support assistant (RAG)

Full design in section 11.

ID	Requirement	P
FR-BOT-1	Product Q&A and recommendations grounded in live catalog data, with product cards in the answer	M
FR-BOT-2	Policy and FAQ answers with source citation	M
FR-BOT-3	Authenticated order-status lookup through a tool call limited to the caller's own orders	S
FR-BOT-4	Streaming responses, conversation memory, handoff to human support (opens a support ticket)	S
FR-BOT-5	Guardrails: refuses out-of-scope requests, resists prompt injection, never reveals other users' data	M

5.13 Customer support messaging and tickets NEW

ID	Requirement	P
FR-SUP-1	Customers message the store through support tickets about an order, a product or their account; threaded conversation with status (open, waiting on customer, resolved), image attachments through the upload pipeline, email/in-app notification of replies; staff inbox with assignment, internal notes and canned replies; chatbot handoff creates a ticket with the conversation attached NEW	S
FR-SUP-2	Support safety and hygiene: per-user rate limits, attachment checks, block abusive users, defined retention period, response-time targets visible to staff, every staff reply audit logged NEW	S

5.14 Storefront experience NEW

ID	Requirement	P
FR-I18N-1	Bangla/English i18n-ready UI: all UI strings externalized from day one, English default with Bangla translation, locale routing, locale-aware number, currency and date formatting, fonts covering Bengali script (section 8.10) NEW	C

ID	Requirement	P
FR-PWA-1	Installable PWA storefront: web app manifest, service worker with offline app shell and offline banner, install prompt on Android and iOS; checkout, account and API responses with personal data are never cached (section 8.11) NEW	C

6. Non-functional requirements

Area	Target
Performance	Listing/product page LCP under 2.5 s on mid-range mobile; API p95 under 300 ms for reads, under 800 ms for checkout
Scalability	500 concurrent checkout attempts on one SKU with zero oversell; stateless API instances behind a load balancer
Availability	99.5% monthly target; graceful degradation if search or chatbot is down (the store still works)
Reliability	All background jobs retriable with backoff and dead-letter queue; all writes that span services use the outbox pattern
Data durability	Daily encrypted backups, point-in-time recovery, restore tested before launch (RPO 15 min, RTO 2 h)
Accessibility	WCAG 2.2 AA for storefront and checkout
SEO	Server-rendered pages, structured data (schema.org Product), sitemap, canonical URLs
Browser support	Last 2 versions of Chrome, Safari, Firefox, Edge; iOS Safari and Android Chrome
Maintainability	Typed code end to end, linting, 80%+ coverage on domain logic, ADR for each major decision
Observability	Request ID on every log line, traces across API and workers, alerts on error rate, latency, queue depth, payment failures
Compliance	Privacy policy, terms, cookie consent, data export/deletion, tax invoices as required locally (GDPR and EU VAT if the target is Hungary/EU)

7. Security requirements

Baseline: OWASP ASVS Level 2 and the OWASP Top 10. Secure by default, deny by default, least privilege. A STRIDE threat model is written for each critical flow (login, checkout, payment webhook, admin uploads, chatbot, support messaging, admin impersonation) in Weeks 1 to 2 and revisited in Week 11 (hardening).

7.1 Authentication and sessions

- Passwords hashed with Argon2id; minimum length 10; check against breached-password list (k-anonymity API); no silly composition rules.
- Access token 10 to 15 minutes; refresh token rotated on every use with reuse detection (reuse revokes the whole session family). Tokens in httpOnly, Secure, SameSite=Lax cookies; CSRF token on state-changing requests.
- Login and reset endpoints rate limited per IP and per account, with exponential backoff and generic error messages (no account enumeration).

- TOTP two-factor mandatory for the super admin and all staff accounts; optional for customers; recovery codes hashed.
- Verification and reset tokens: 256-bit random, stored hashed, single use, short expiry. Phone OTP codes: short expiry, stored hashed, limited attempts, rate limited per phone number and per IP.

7.2 Authorization

- One central policy layer; every endpoint declares required role and ownership rule; deny by default.
- Object-level checks on every read and write (user_id scoping in the query itself, explicit staff permissions on every admin endpoint) to prevent IDOR. An automated test suite tries cross-user access and staff-permission escalation on every endpoint.
- Sensitive admin actions (refund, role or permission change, stock write-off, configuration change) require recent re-authentication and are audit logged.
- Admin impersonation (FR-ADM-8) uses a separate, time-limited impersonation session that records actor and subject on every request; read-only by default and blocked from credential, 2FA and payment actions.

7.3 Input, output and files

- All input validated by Pydantic (server) and Zod (client); database access only through parameterized queries.
- Output encoded by default; rich-text product descriptions sanitized with an allowlist (DOMPurify on render, server-side sanitization on save). Support messages are plain text, encoded on render.
- Uploads: size and type limits, content sniffing, images re-encoded and EXIF stripped, random file names, private bucket with short-lived signed URLs, optional antivirus scan for documents. Support ticket attachments use the same pipeline.
- The server never fetches user-supplied URLs (prevents SSRF); any exception uses an allowlist.

7.4 Transport and browser hardening

- TLS 1.2+ only, HSTS with preload, nonce-based Content-Security-Policy, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, frame-ancestors 'none'.
- CORS allowlist of known origins only; no wildcard with credentials.
- Cookies prefixed (__Host-) where possible; subresource integrity for any third-party script; minimal third-party scripts. The PWA service worker is served from the site origin with a strict scope.

7.5 Payments and business-logic abuse

- Hosted gateway only; card numbers, CVV and PINs are never received, logged or stored (keeps PCI scope minimal).
- Prices, discounts, delivery charge and totals are computed on the server only; the client total is display-only.
- Every payment is verified server-side with the gateway's validation API (amount, currency, order reference, status) before the order is marked paid. Webhooks are signature checked and idempotent: processed effectively once via deduplication on the gateway event id (unique constraint).
- **Gateway return and callback endpoints never depend on session cookies.** Gateways often send the browser back with a cross-site POST, and SameSite=Lax cookies are not sent on that request. These endpoints identify the payment by an opaque, unguessable reference, verify its state only through the

gateway validation API (never from redirect parameters), and then respond with a 303 redirect to an order-status page that loads normally with the session.

- Late confirmations (payment success after expiry or cancellation) follow section 9.5: re-reserve and reinstate, or auto-refund and notify.
- Abuse controls: coupon usage limits per user and device, reservation limits per user to stop stock hoarding, COD risk rules (OTP confirmation, order-count limits, blocklist) driven by the fraud risk score (FR-PAY-7), review-fraud checks, bot protection (CAPTCHA/Turnstile) on signup, login and checkout.

7.6 Data protection and privacy

- Encryption in transit and at rest (database, backups, object storage). Field-level encryption for sensitive data (2FA secrets, bank or mobile-wallet details collected for COD refunds).
- Collect the minimum personal data; defined retention periods (including support tickets and risk signals); export and deletion on request; consent record for marketing.
- No secrets, tokens, full emails or phone numbers in logs; log scrubbing tested.
- Secrets in a secret manager or encrypted environment store, never in the repository; keys rotated on a schedule.

7.7 Infrastructure and supply chain

- Database not publicly reachable; separate least-privilege DB users for app, migrations, and read-only reporting.
- WAF/CDN in front (rate limiting, bot rules); app-level rate limits in Redis.
- CI gates: dependency audit (pip-audit, npm audit, Dependabot), SAST (Bandit, CodeQL), secret scanning (gitleaks), container scan (Trivy), SBOM generated per release.
- Containers run as non-root with read-only filesystem where possible; images pinned by digest; infrastructure defined as code.

7.8 AI-specific security

- Retrieved text (product descriptions, reviews, documents) is treated as untrusted data, never as instructions (defence against indirect prompt injection).
- Tool calls run with the caller's own permissions, validated server-side; the model can never choose a user id.
- Output filtering for secrets and PII; per-user rate limits and cost caps; conversation logs redacted.
- A jailbreak and injection test set is part of the evaluation suite and runs in CI.

7.9 Monitoring and incident response

- Immutable audit log for admin, staff and payment actions (including impersonation sessions and feature-flag changes); alerts on login anomalies, spikes in 4xx/5xx, webhook failures, refund spikes, late-payment auto-refunds.
- Written incident runbook (contain, assess, notify, recover, review) and a restore drill before launch.

7.10 Security acceptance gate (before real payments)

1. Authorization test suite green (no IDOR on any endpoint).
2. OWASP ZAP baseline and authenticated scans clean of high findings.
3. Dependency and container scans clean of high/critical issues.

4. Payment sandbox tests: duplicate webhook, forged webhook, amount tampering, replay, timeout, late confirmation after reservation expiry or cancellation (reinstate and auto-refund paths), and gateway return without a session cookie.
5. Backup restore tested.
6. Independent review by a second person. Because this system will handle real money and customer data, a third-party penetration test is strongly recommended before significant volume. This document is a plan, not a security certification.

8. UI and UX requirements

8.1 Principles

Mobile-first (most e-commerce traffic is on phones), fast, trustworthy, and clear. Every screen answers: where am I, what can I do, what happens next.

8.2 Design system

- Design tokens for colour, typography, spacing, radius, elevation and motion; light and dark themes; brand palette chosen for contrast (all text at least 4.5:1).
- Component library built on shadcn/ui and Radix, documented in Storybook: buttons, inputs, select, combobox, dialog, drawer, toast, tabs, table, pagination, badge, rating, price, product card, skeletons, empty states.
- Icons from one set (Lucide); motion kept subtle and disabled under prefers-reduced-motion.
- A single design file (Figma) with the key flows designed before build, in Weeks 1 to 2.

8.3 Key screens

Area	Screens
Storefront	Home (banners, categories, deals, recommendations), category listing with filters, search results, product detail (with price history, Could), cart, checkout, order success, order tracking
Account	Login/register, verification (email link or phone OTP), profile, addresses, orders and returns, support tickets, wishlist, reviews, notification settings, language
Admin back office	Dashboard and sales analytics, products and variants, bulk upload, inventory and stock adjustments, orders and fulfilment (pick, pack, ship), returns and returned parcels, refunds, support inbox, review and Q&A moderation, risk review queue, customers, staff and permissions, catalog and homepage, coupons, delivery zones, feature flags, settings, audit log, impersonation (Could)
Assistant	Floating chat widget, full-screen on mobile

8.4 Checkout UX rules

- Stepped but single-page: address, delivery, payment, review. Progress visible, back never loses data.
- Inline validation, address autocomplete, saved addresses, phone-number formatting.
- Total, delivery charge, discount and tax always visible and consistent with the final charge.
- Payment pending, failed, and abandoned states have clear messages and a safe retry; the cart is never lost. A late payment that is refunded automatically is explained clearly to the customer.
- Trust cues near the pay button: secure payment note, return policy link, support contact.
- If the account has no verified email or phone, checkout asks for verification inline (FR-AUTH-1) without losing the cart.

8.5 States and feedback

Every data view has loading skeletons, an empty state with a next action, and an error state with retry. Cart and wishlist use optimistic updates with rollback. Network loss shows a banner, not a blank page.

8.6 Accessibility (WCAG 2.2 AA)

Keyboard operable everywhere with visible focus; semantic landmarks and headings; form labels and error association; focus management in dialogs and drawers; alt text for product images; no information by colour alone; tap targets at least 44 px; screen-reader test of checkout; automated axe checks in CI.

8.7 Performance budget

- Per-route JavaScript under 170 KB gzipped on storefront pages; Lighthouse performance 90+ on mobile for home, listing and product pages.
- next/image with responsive sizes and AVIF/WebP; fonts self-hosted and subset (including a Bengali subset when Bangla is enabled); ISR with revalidation for product and category pages; no layout shift (reserved image and banner space).
- Third-party scripts loaded after interaction. The price-history chart is lazy-loaded below the fold.

8.8 Chat widget UX

Streaming answers; product cards with image, price, stock and add-to-cart; visible source citations for policy answers; thumbs up/down feedback; 'talk to a person' handoff; a short privacy notice; keyboard accessible; works on small screens.

8.9 Content and SEO

Clear microcopy and error messages (what happened and what to do), schema.org Product markup, Open Graph tags, sitemap and robots, canonical URLs for filtered listings, hreflang tags once more than one locale is live.

8.10 Internationalization (FR-I18N-1) NEW

- No hard-coded UI strings from Week 1: all copy lives in message catalogs (English default, Bangla added later) with ICU plural rules.
- Locale in the URL (for example /en, /bn) with a language switcher and remembered preference; email and SMS templates are localized too.
- Formatting through Intl APIs: currency (for example BDT, EUR or HUF), numbers, dates and time zones; layouts tolerate longer strings; fonts cover Bengali script (for example Noto Sans Bengali).
- Product content stays single-language in v1; translatable product fields are a later extension.

8.11 Progressive Web App (FR-PWA-1) NEW

- Web app manifest (name, icons, theme colour, standalone display) and an install prompt shown after engagement, never on the first visit.
- Service worker caches the app shell and static assets; catalog pages use stale-while-revalidate; offline shows a branded page and the network-loss banner.
- Never cached: checkout, account, payment return pages and any API response containing personal data; the service worker is versioned and updates safely.
- Web push notifications are out of scope for v1 (possible later with FR-NOT-3).

9. Architecture and data model

9.1 System overview

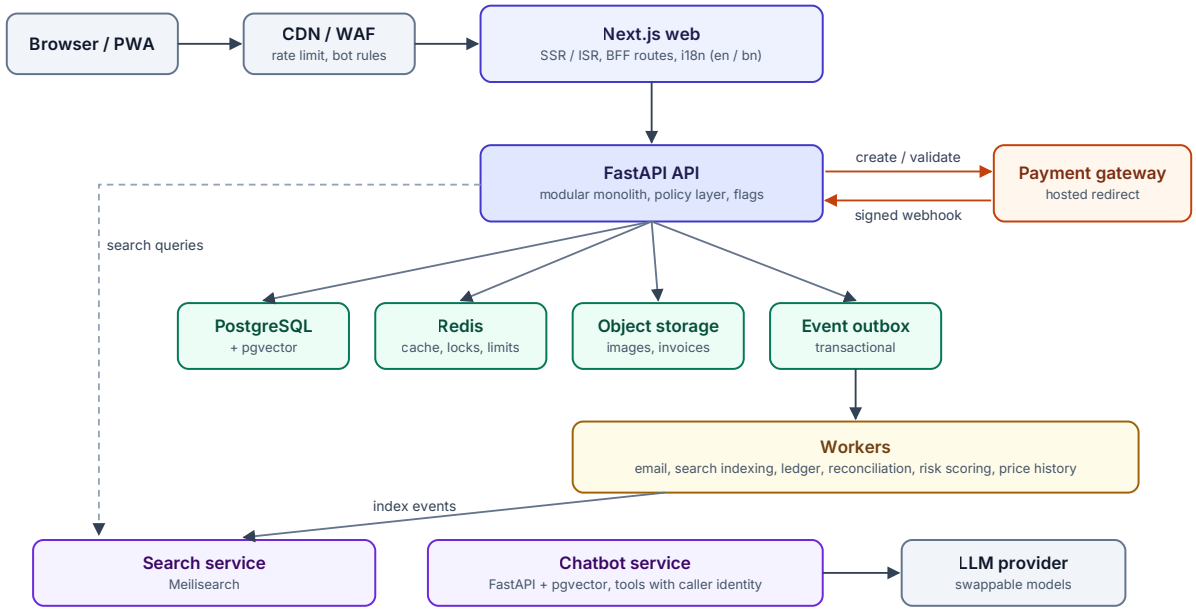


Figure 1. System overview: browser to CDN/WAF to Next.js, the FastAPI modular monolith, data tier, workers, search and chatbot services, and the payment gateway.

9.2 Module rules

Modules: auth, users, catalog, inventory, cart, order, payment, shipping, review, promotion, notification, search, chatbot, admin, audit, support, risk and flags.

- Each module owns its tables and exposes a service interface; other modules never write to its tables.
- Cross-module reactions use domain events (OrderPlaced, PaymentConfirmed, StockReserved, ProductUpdated, PriceChanged, OrderReinstated, RefundIssued, TicketCreated) delivered through a transactional outbox so a state change and its event commit together.
- Layers inside a module: router, service (business rules), repository (queries), schemas. Business rules never live in routers.
- Repository layout: apps/web (Next.js storefront and admin back office), apps/api (FastAPI), apps/worker, services/chatbot, packages/api-client (generated), infra, docs/adr.

9.3 Core entities

Entity	Notes
users, roles, permissions, user_roles, sessions, addresses	Argon2id hash, 2FA secret (encrypted), refresh-token families; email_verified_at and phone_verified_at; phone OTP challenges (hashed, short expiry); staff permissions per account
categories, attributes, brands, collections	Tree with materialized path

Entity	Notes
products, product_variants (SKU), product_images	Publishing workflow, slug, search document
price_history NEW	Append-only per variant (price, compare-at price, valid_from); 30-day lowest price query
inventory, inventory_reservations, stock_movements	CHECK (available >= 0); reservation expires_at; every adjustment, receipt, sale, restock and write-off recorded with reason and actor
carts, cart_items	Price snapshot per line
orders, order_items, order_events	One order per checkout with a single state machine and refund_status; order_items carry refunded_qty and refunded_amount; append-only events
payments, payment_attempts, webhook_events	Unique idempotency key and gateway reference; UNIQUE (gateway, gateway_event_id) on webhook_events for deduplication
refunds, returns, return_items	Linked to order items; partial amounts and quantities; rejected status with reason
ledger_entries	Simple revenue and refund ledger: append-only entries (payment captured, COD collected, refund, gateway fee) per order, never updated
coupons, coupon_redemptions	Per-user and global limits
reviews, review_images, wishlists, questions	Verified-purchase link
support_tickets, ticket_messages, ticket_attachments NEW	Ticket per customer, optionally linked to an order or product; status, assignee, internal notes; retention policy
risk_assessments, risk_rules, blocklist_entries NEW	Score, decision and reasons per checkout/order; versioned rules; phone, address, device and email blocklist
feature_flags, feature_flag_overrides NEW	Key, type, rollout percentage, targeting rules; changes audit logged
impersonation_sessions NEW	Actor, subject, consent reference, scope (read-only), started_at, expires_at, ended_at
notifications, notification_preferences	Channel and consent; preferred locale
audit_logs, outbox_events	Immutable, indexed by actor and entity (and impersonated subject when present)
chat_conversations, chat_messages, knowledge_documents, knowledge_chunks (vector)	Embedding model and version stored

Data rules: money stored as integer minor units plus currency code (never floats); UUIDv7 primary keys; all timestamps UTC; soft delete where history matters; foreign keys and check constraints enforced; indexes designed from real query plans (EXPLAIN reviewed for listing, search, checkout and order queries); every migration reversible and reviewed.

9.4 Order state machine UPDATED

There is one order per checkout and one state machine at the order level. Payment progress and fulfilment progress are both expressed as order states; refund_status (none, partial, full) is tracked alongside, because partial refunds do not change the fulfilment state.

Table 9.4: Order transitions

From	Event	To
pending_payment	Online payment confirmed (signature + validation API)	paid
pending_payment	COD accepted (risk rules and OTP passed)	confirmed
pending_payment	Timeout, payment failed, user cancel	cancelled (stock released)
cancelled	Late verified payment, all stock re-reserved (9.5)	paid (order reinstated)
cancelled	Late verified payment, stock unavailable (9.5)	cancelled (automatic full refund, customer notified)
paid	Staff confirms (or automatic after risk check)	confirmed
confirmed	Warehouse packs	packed
packed	Handed to courier	shipped
shipped	Courier delivered (COD: cash collected)	delivered
shipped	Courier failed delivery or COD refused	delivery_failed
delivery_failed	Re-delivery attempt (within the configured maximum)	shipped
delivery_failed	Parcel sent back to the warehouse	returned
returned	Warehouse confirms receipt	cancelled (stock restocked; prepaid amount refunded)
delivered	Return window ends	completed
delivered	Customer requests return	return_requested
return_requested	Approved and received, all lines	refunded
return_requested	Approved and received, some lines	completed (partial refund on those order items)
return_requested	Rejected with reason	return_rejected
return_rejected	Customer notified; dispute window ends or complaint closed	completed
paid / confirmed	Cancel before shipping	cancelled (stock released; refund triggered if prepaid)

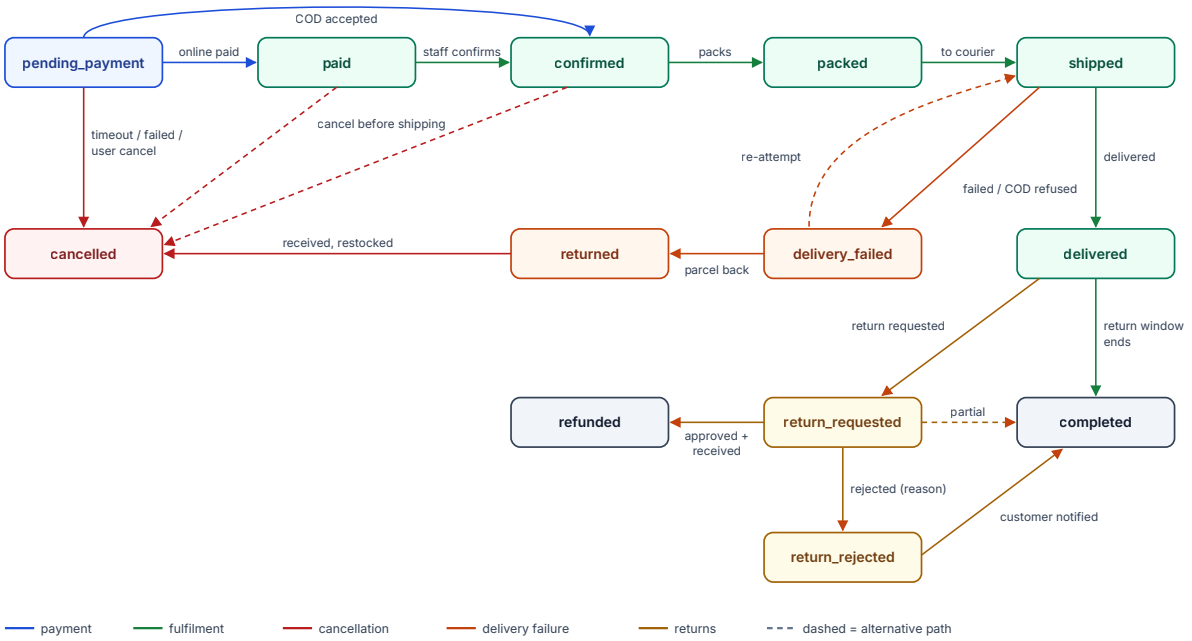


Figure 2. Order state diagram (late-payment reinstatement from cancelled to paid is shown in Figure 3).

Partial refunds. Refunds are recorded per order item (quantity and amount) and roll up to the order. Refunds can never exceed the captured amount; every refund writes a ledger entry. refund_status moves from none to partial to full.

Only transitions in this table are allowed; each is a single database transaction that writes the new state, an order_events row and an outbox event.

9.5 Checkout sequence

1. Client sends checkout request with an Idempotency-Key.
2. Server loads cart, re-prices every line, validates coupon and delivery zone, checks that the user has a verified email or phone, and computes the fraud risk score (FR-PAY-7).
3. In one transaction: reserve stock atomically per SKU, create the order in pending_payment, write outbox event.
4. Create payment with the gateway (or mark COD pending confirmation) and return the redirect URL. The return URL carries only an opaque payment reference.
5. Gateway webhook arrives; verify signature; insert into webhook_events (a duplicate gateway event id is ignored); call the gateway validation API; in one transaction mark payment and order paid, convert reservation to a sale, write a ledger entry and events. The browser return endpoint does the same verification without relying on cookies, then redirects (303) to the status page.
6. Worker sends confirmation email, updates search stock flag, notifies warehouse staff.
7. A scheduled job releases expired reservations and cancels unpaid orders.
8. If a verified payment arrives after step 7 for that order, the late-confirmation flow below runs.

Late payment confirmation (FR-PAY-8) NEW

A gateway can confirm a payment after the reservation expired or the order was cancelled (slow customer, delayed webhook, reconciliation finding). Such a payment is never ignored and never silently kept.

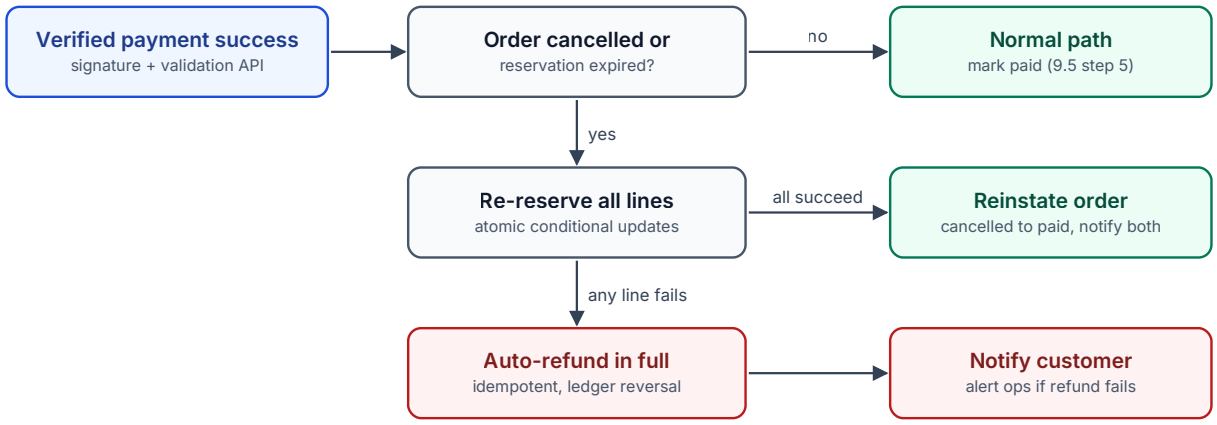


Figure 3. Late payment confirmation flow.

- **Re-reserve:** in one transaction, try the atomic conditional stock update for every line. If every line succeeds, reinstate the order (cancelled to paid), write an OrderReinstated event, and notify the customer and warehouse staff.
- **Refund:** if any line fails, roll back the re-reservation, keep the order cancelled, issue an automatic full refund through the gateway with an idempotency key, record ledger entries, and notify the customer with a clear explanation. A failed refund raises an alert and lands in the admin refund queue.
- **Prices:** the captured amount stands; a reinstated order keeps the prices that were paid.
- **Tests:** integration tests cover both branches, a duplicate late webhook, a late webhook racing the expiry job, and reconciliation finding a late payment; they belong to the release-blocking payment suite.

9.6 Inventory without overselling

Stock is decremented with a single conditional statement (`UPDATE inventory SET available = available - :qty WHERE sku_id = :id AND available >= :qty`); zero rows updated means out of stock. Reservations carry an expiry. For flash sales, a Redis counter fronts the database and is reconciled. The load test (k6) proves it: N concurrent buyers on a stock of M must produce exactly M successful reservations. Restocking after a returned parcel, an accepted return or a cancellation goes through the same table and writes a stock_movements row.

9.7 Caching

Cache-aside in Redis for category tree, homepage sections and product detail fragments; invalidation by ProductUpdated and PriceChanged events; ISR on the web tier; stampede protection with short locks; cart and rate-limit counters live in Redis only. Feature-flag evaluations are cached for a few seconds and invalidated on change.

10. API conventions

- REST under /api/v1; nouns for resources, plural names, consistent verbs; OpenAPI generated by FastAPI and published to a typed TypeScript client (openapi-typescript) used by the web app, so contract drift fails the build.
- Cursor-based pagination for lists; filtering and sorting through documented query parameters; response envelope kept minimal.
- Errors use RFC 9457 problem details (type, title, status, detail, request id); validation errors list fields.
- Idempotency-Key header required on checkout, payment creation and refund requests; keys stored with the response for 24 hours.
- Rate limit headers (limit, remaining, reset) on every response; stricter limits on auth, OTP, search, support tickets and chatbot.
- Backwards-compatible changes only inside a version; deprecations announced with a Sunset header.
- Gateway return (browser) and callback endpoints are session-independent: no cookie or CSRF dependency, identified by an opaque payment reference, verified only through the gateway validation API, and safe to call repeatedly.

Group	Main endpoints
Auth	register, verify-email, verify-phone (send OTP, confirm OTP), login, refresh, logout, password reset, 2FA setup and verify, sessions
Catalog	categories, products (list, detail, by slug), brands, collections, price history per variant (Could)
Search	search, suggestions, facets
Cart and checkout	cart (get, add, update, remove, merge), coupons apply, checkout quote, checkout submit
Orders	my orders, order detail, cancel, return request, tracking, invoice
Payments	create payment, gateway return (session-independent, 303 to status page), gateway callback, webhook receiver (unauthenticated but signed, deduplicated by event id), refund status
Support NEW	tickets (list, create for an order, product or account), messages (list, send), attachments, close or reopen
Admin	products, variants and images CRUD, inventory and stock adjustments, orders and fulfilment status updates (including delivery failed and returned), return decisions, refunds, ledger and sales analytics, support inbox, customers, staff and permissions, review moderation, coupons, delivery zones, settings, audit log, risk review queue and rules, feature flags, impersonation start and end (Could)
Reviews	list, create (verified only), report, moderate
Flags NEW	evaluated client-safe flags for the current user (read-only)
Assistant	conversations, send message (streaming), feedback, handoff

11. RAG assistant design

11.1 Capabilities

1. Product assistant: understands requests like 'good headphones under 5000, long battery', queries the catalog, and answers with product cards.
2. Policy and FAQ assistant: answers returns, shipping, payment and account questions from approved documents, with citations.
3. Order support (tool use): after login, reports status of the caller's own orders and explains next steps; can open a support ticket.

Each capability has its own prompt, tools and evaluation set; a lightweight router classifies the message first.

11.2 Knowledge ingestion

- Sources: product catalog (title, description, attributes, price, stock, rating), policy pages, FAQs, help articles.
- Product data is chunked by product; documents by heading with overlap; metadata kept (category, price, brand, in_stock, language, source, updated_at).
- Embeddings stored in pgvector with the model name and version; re-index triggered by ProductUpdated and document-change events; nightly full consistency check.
- Price and stock are never trusted from the vector store: they are re-read from the database at answer time.

11.3 Retrieval and generation

- Hybrid retrieval: keyword (BM25 or Meilisearch) plus vector similarity, merged with reciprocal rank fusion, then a cross-encoder reranker; metadata filters applied from parsed constraints (price, category, in stock).
- Query rewriting using conversation history; if retrieval confidence is low the assistant says it does not know and offers human support, instead of guessing.
- The prompt separates system rules, retrieved context (marked as untrusted data) and the user message; answers must cite sources; structured output for product cards.
- Streaming through server-sent events; timeouts, retries and a fallback response if the LLM provider fails. The assistant sits behind a feature flag (kill switch).

11.4 Guardrails

- Scope limiter (shopping and support only), prompt-injection and jailbreak defences (see 7.8), PII redaction, refusal templates, per-user rate limit and monthly cost cap, content moderation on input and output.
- Tools are allowlisted, take validated arguments, and run with the caller's identity.

11.5 Evaluation (a senior-level signal)

- A curated test set of at least 100 questions across the three capabilities, including adversarial and unanswerable questions.

- Metrics: retrieval hit rate at k, answer faithfulness (grounded in context), answer relevance, refusal accuracy, injection resistance, latency (p50/p95) and cost per conversation.
- Evaluation runs in CI on prompt, model or retrieval changes; results are tracked over time and summarized in the README.
- Production feedback (thumbs, handoffs) feeds the next iteration of the test set.

12. DevOps, environments and observability

- Environments: local (Docker Compose), staging (production-like, seeded), production. Config through environment variables; no environment-specific code paths. Feature flags (FR-ADM-7) separate deploy from release and act as kill switches.
- CI: lint, type check (mypy, tsc), unit and integration tests, migration check, build images, security scans, Playwright smoke tests on a preview deployment. CD: automatic to staging, manual approval to production, one-command rollback, database migrations backwards compatible (expand then contract).
- Deployment: containers on a VPS or managed platform behind a reverse proxy with automatic TLS; Next.js on Vercel or self-hosted; Kubernetes only if justified in an ADR.
- Observability: OpenTelemetry traces across web, API and workers; Prometheus metrics and Grafana dashboards (request rate, errors, latency, queue depth, checkout funnel, payment success, late-payment reinstatements and auto-refunds, stock reservation failures, risk decisions, LLM latency and cost); Sentry for exceptions; structured JSON logs with request IDs.
- Alerts: error-rate and latency SLO burn, payment webhook failures, failed automatic refunds, queue backlog, reservation expiry job lag, disk and database health.
- Backups and recovery: daily encrypted backups with point-in-time recovery, monthly restore drill, documented runbooks.

13. Testing strategy

Level	Tools	What it covers
Unit	pytest, Vitest	Pricing, coupons, order state machine, permission rules, refund calculation and ledger entries, risk rules
Integration	pytest with a real PostgreSQL and Redis in containers	Repositories, transactions, outbox, webhooks, idempotency, event-id deduplication, late webhooks (reinstate and auto-refund paths), session-less gateway return
Contract	OpenAPI diff and generated client type checks	Frontend and backend stay in sync
End to end	Playwright	Register (email and phone OTP), search, add to cart, checkout (sandbox payment), staff fulfils (pick, pack, ship), COD delivery failure and returned parcel, return and partial refund, support ticket
Security	Authorization matrix tests, ZAP, dependency scans	Section 7.10 gate, including staff-permission and impersonation restrictions
Load	k6	Browse traffic, search, 500 concurrent checkouts on one SKU, webhook bursts
Accessibility and performance	axe, Lighthouse CI	Section 8.6 and 8.7 budgets
AI evaluation	Custom harness (Ragas-style metrics)	Section 11.5

Target at least 80% coverage on domain logic (pricing, orders, payments, inventory); failures in payment and inventory tests (including late-webhook tests) block release.

14. Twelve-week plan UPDATED

The single-vendor scope is smaller than the earlier plan assumed, so the build is planned for 12 weeks, including buffer in Week 12. Assumes roughly 40 to 50 focused hours per week. Each week ends with a demo on staging and updated docs.

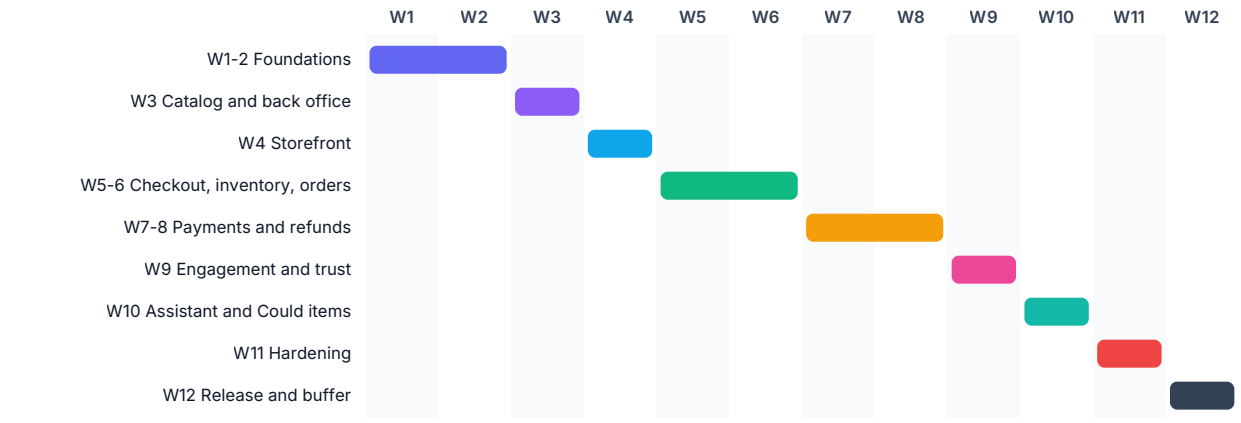


Figure 4. Twelve-week timeline.

Week	Focus	Exit criteria
1-2	Foundations	Auth (register, login, refresh rotation, email verification, phone OTP verification), 2FA (mandatory for super admin and staff), RBAC with staff permissions, CI, Docker Compose and monorepo, threat model v1, Figma flows, design tokens, audit log, i18n-ready string setup, first ADRs
3	Catalog and back office	Categories, products, variants, image pipeline, publishing workflow, inventory and stock movements, admin back office shell, search indexing pipeline
4	Storefront	Home, listing, product page, search with filters and autocomplete, guest and user cart, SEO basics, Lighthouse budget in CI
5-6	Checkout, inventory, orders	Atomic stock reservation, order creation, order state machine (including delivery_failed and returned), COD flow, back-office fulfilment (confirm, pack, ship, tracking), tracking page, transactional emails, first k6 oversell test
7-8	Payments and refunds	Gateway integration, signed webhooks, idempotency and event-id deduplication, session-less return endpoint, late-webhook handling, reconciliation job, returns and partial refunds, revenue/refund ledger, invoices
9	Engagement and trust	Coupons, verified reviews, wishlist, notifications, customer-support tickets, COD/new-account fraud scoring, feature flags
10	Assistant and Could items	RAG assistant (ingestion, hybrid retrieval, tools, chat widget, 100-question eval set), admin sales analytics; price history, PWA, Bangla translation and admin impersonation if time allows

Week	Focus	Exit criteria
11	Hardening	Security gate (7.10), load tests, observability dashboards and alerts, accessibility audit, performance tuning
12	Release and buffer	Staging soak test, restore drill, production deploy, seed and demo data, README, architecture diagrams, ADR cleanup, demo video; about half of the week reserved as buffer for slippage

Scope cut order if time runs short: first all Could items (admin impersonation, Bangla translation (keep the UI i18n-ready), PWA, price history, recommendations, product Q&A, in-app notifications, guest checkout, flash sales), then Should items in this order: bulk CSV import, SMS (keep OTP for phone-registered users, or limit registration to email), return photos, sales analytics detail, support-ticket extras (keep basic tickets with email replies), advanced fraud-scoring signals (keep the basic COD rules), feature-flag targeting (keep simple on/off kill switches), ledger reporting (keep the append-only ledger entries). Never cut: authorization tests, payment verification, idempotency, oversell protection, backups, accessibility basics.

15. Risks and mitigations

Risk	Impact	Mitigation
Scope too large for 12 weeks	Half-finished features	MoSCoW priorities, weekly exit criteria, cut order above, no new features after Week 10, Week 12 buffer
Payment gateway onboarding delay	Cannot go live	Start merchant onboarding in Week 1; develop against sandbox; COD as fallback
Target country or gateway undecided	Payments work in Weeks 7 to 8 blocked or redone	Decide the country first (section 2); gateway adapter interface; confirm owner and entity eligibility before Week 1
Overselling or double charge	Financial loss, trust loss	Atomic stock updates, idempotency, event-id deduplication, reconciliation, load and failure tests
Late or out-of-order webhooks	Paid but cancelled orders, unhappy customers	Late-confirmation flow (re-reserve or auto-refund), reconciliation, alerts, dedicated tests
Refund or ledger errors	Money lost or wrongly refunded	Refund caps per order item, append-only ledger, ledger-to-gateway reconciliation, invariant tests
COD fraud and refusals	Lost delivery costs, stock tied up	Risk scoring, OTP, order limits, blocklist, delivery_failed and returned flow
Staff misuse or account takeover	Data leak, fraudulent refunds	Least-privilege staff permissions, mandatory 2FA, refund limits, audit log, audited impersonation
Security incident	Data loss, legal exposure	Section 7 gate, third-party review, least privilege, backups, incident runbook
Support load at launch	Slow replies, lost sales	Chatbot answers common questions, ticket inbox with canned replies, response-time targets
LLM cost or abuse	Unexpected bills	Rate limits, cost caps, caching, small model for routing, feature-flag kill switch
Hallucinated answers	Wrong prices or policies	Ground in data, re-read price and stock from DB, citations, refusal when unsure, evaluation in CI
Single-developer bus factor	Hard to maintain	ADRs, runbooks, README, conventions, automation
Legal and tax compliance	Fines or blocked payments	Confirm local requirements with an accountant or lawyer before launch

16. Definition of done and launch checklist

Per feature: acceptance criteria met; unit and integration tests; authorization test; accessibility check; metrics and logs added; docs and API schema updated; reviewed (self-review checklist or second reviewer); risky features shipped behind a feature flag.

Launch checklist

☐ Payment sandbox scenarios pass (success, failure, duplicate, forged, timeout, partial refund, late confirmation after expiry, return without session cookie)

☐ Live gateway credentials configured and a small real transaction verified, then refunded

☐ Security gate in 7.10 complete; independent review done

☐ Backup restore drill completed; RPO and RTO confirmed

☐ Load test report attached (zero oversell at target concurrency)

☐ Monitoring dashboards and alerts live; on-call contact defined

☐ Terms, privacy policy, return policy, cookie consent published and reviewed

☐ Legal entity, tax and invoice format confirmed for the target country

☐ Staging soak test of at least 3 days with synthetic traffic

☐ Rollback procedure rehearsed

☐ Super admin and staff accounts created with 2FA enforced and least-privilege permissions; default credentials removed

☐ Support process defined (email, phone, chatbot handoff, support tickets)

17. Portfolio and CV plan

- Repository: public or private-with-access README with architecture diagram, feature list, screenshots, live demo link with a demo account, how to run locally in one command.
- Architecture Decision Records (docs/adr), for example: modular monolith over microservices; PostgreSQL plus pgvector over a separate vector database; outbox pattern; hosted-gateway payments; cursor pagination; Meilisearch versus OpenSearch; hybrid retrieval with reranking; order state machine and late-payment handling.
- Evidence section: load-test results (zero oversell at N concurrent checkouts), p95 latencies, Lighthouse scores, test coverage, RAG evaluation table, security gate results.
- Write-ups (optional but strong): 'How the checkout avoids overselling', 'Evaluating a RAG shopping assistant', 'Designing idempotent payment webhooks'.
- CV bullet pattern: 'Designed and built a single-vendor e-commerce platform (Next.js, FastAPI, PostgreSQL, Redis) serving real customers; prevented overselling at 500 concurrent checkouts; reduced LCP to under 2.5 s; shipped a RAG assistant with measured 0.9 faithfulness'. Use only numbers you actually measured.

18. Next steps

1. Confirm the open decisions in section 2, starting with the target country (it determines gateway, currency, tax and legal entity), then currency, gateway, legal entity and owner eligibility, returns and delivery rules, COD, budget.
2. Create the repository and the Weeks 1 to 2 backlog from section 14.
3. Produce the detailed ERD, API specification and folder structure as follow-up documents.
4. Start merchant onboarding with the payment gateway immediately; it often takes the longest.